

实验报告

实验 2 组合逻辑部件设计

课程名称

数字逻辑与计算机组成

指导教师

吴海军

1 实验目的

- 掌握译码器、编码器、数码管等组合部件的设计方法。
- 掌握 1 位加法器、4 位串行加法器的设计方法。
- 掌握补码减法运算方法，实现补码减法功能。
- 掌握汉明码编码和纠错的设计方法。
- 掌握桶形移位器的设计方法。

2 实验环境

- 平台：Logisim 2.16

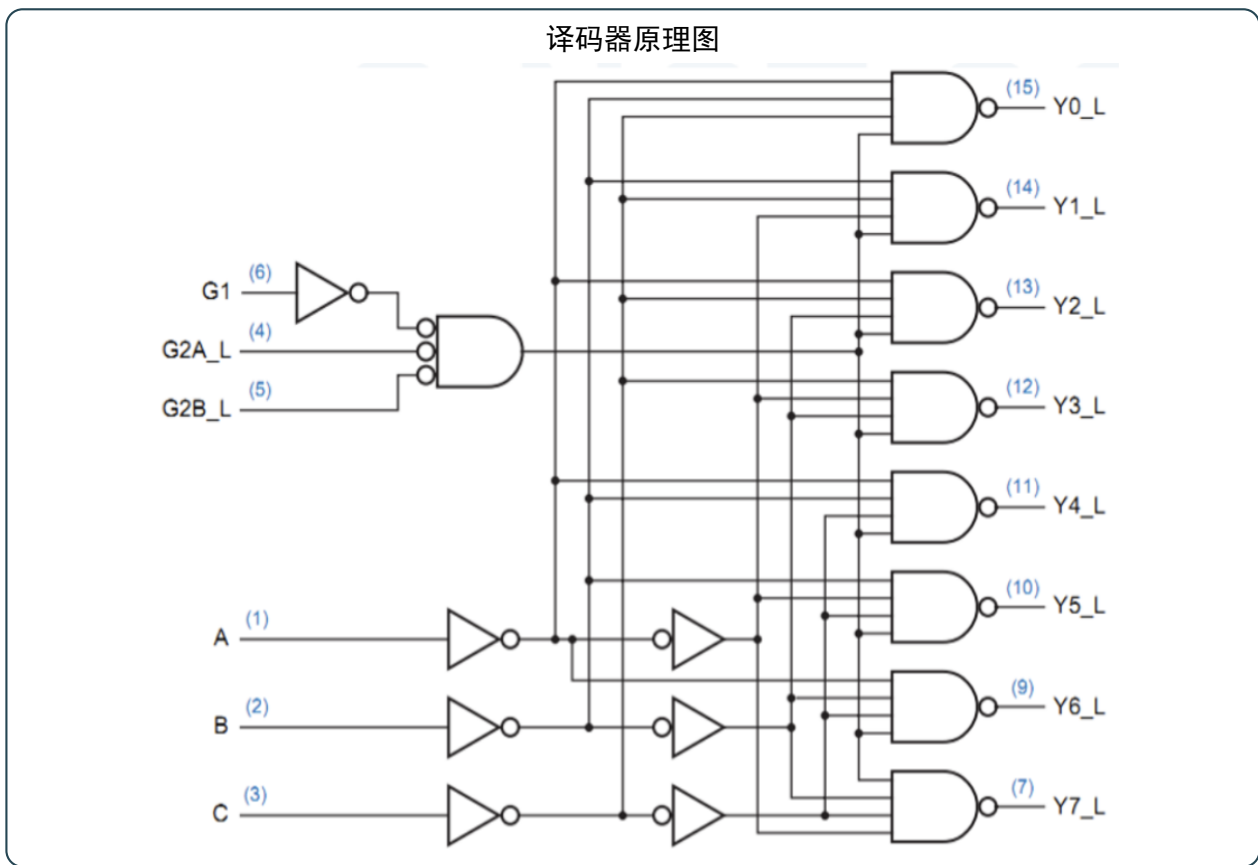
3 实验内容

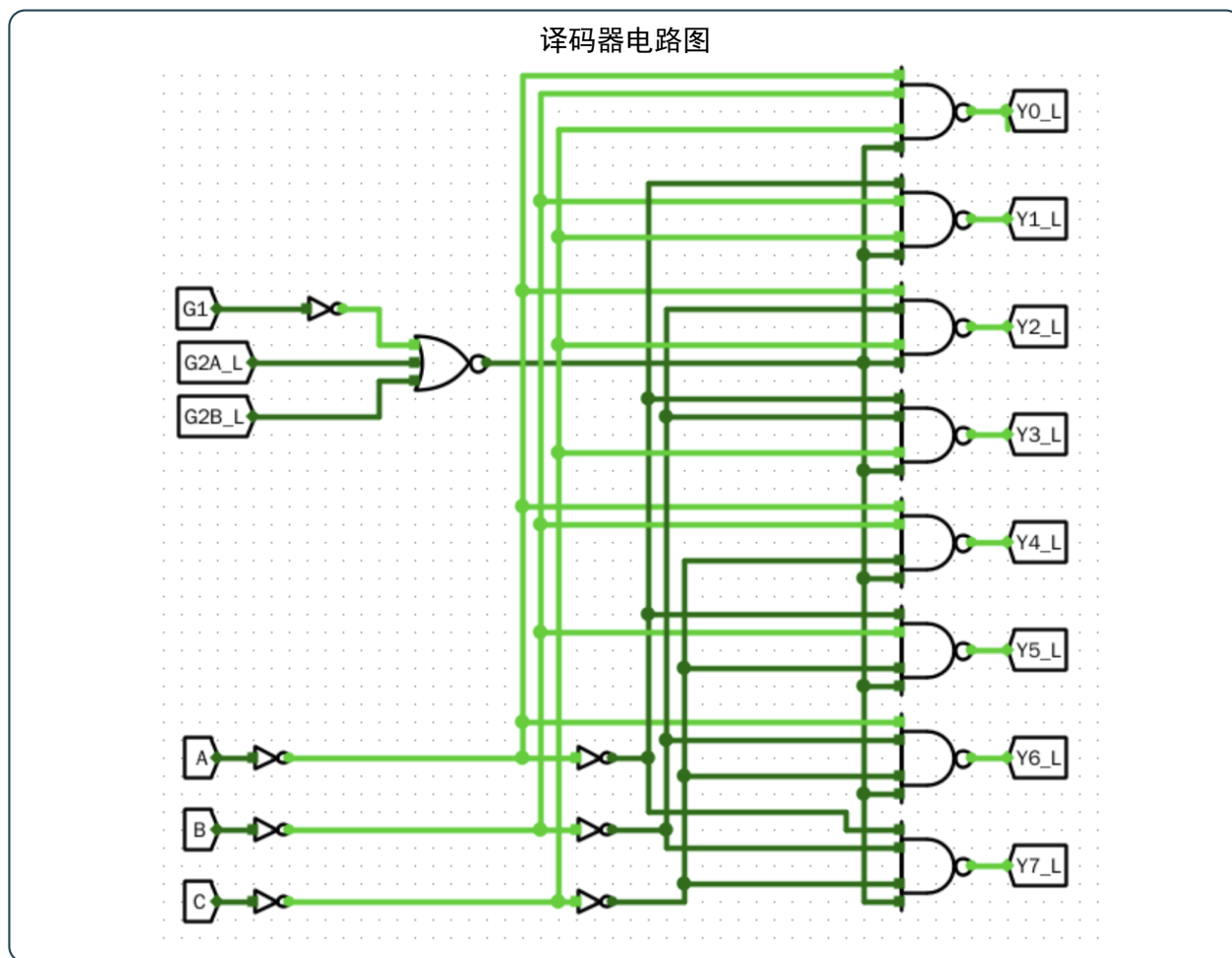
3.1 译码器实验

3.1.1 实验整体方案设计

- 顶层模块设计：实验电路较为简单，不需要顶层模块设计图。
- 输入输出引脚：G1、G2A_L、G2B_L 为译码器使能端 (G1 高电平有效，G2A_L、G2B_L 低电平有效)，ABC 3-8 为译码器的输入端 (C 为最高位，A 为最低位)，Y1_L 到 Y8_L 为 3 译码器输出端 (均为低电平有效)。
- 实现思路：构造优先级判断与编码输出。

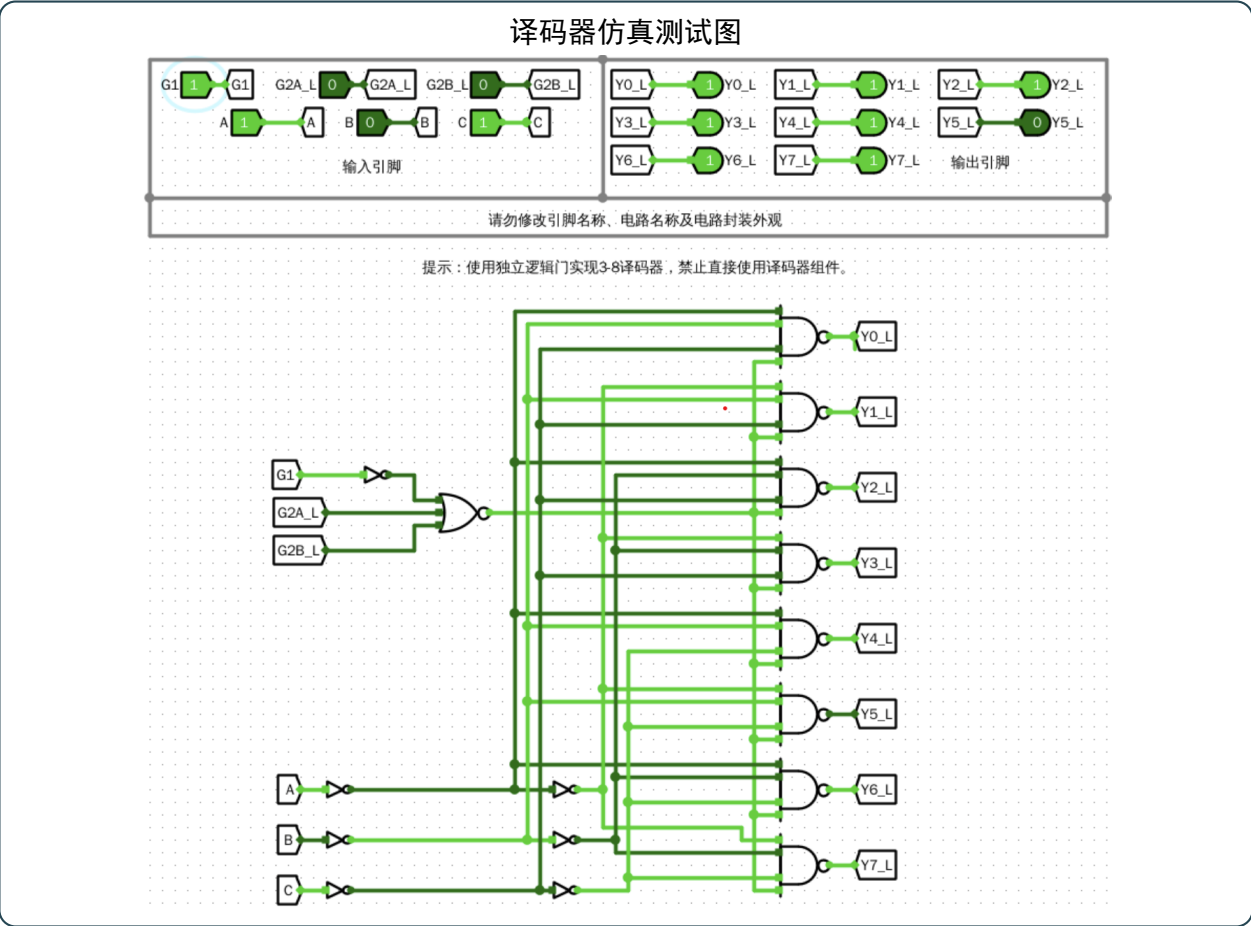
3.1.2 实验原理图和电路图





3.1.3 实验数据仿真测试图

线上自动测试通过，下面是对某输入的仿真测试截图。



74X138 功能表													
输入						输出							
G1	G2A_L	G2B_L	C	B	A	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0
X	1	X	X	X	X	1	1	1	1	1	1	1	1
X	X	1	X	X	X	1	1	1	1	1	1	1	1
0	X	X	X	X	X	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1	0
1	0	0	0	0	1	1	1	1	1	1	1	0	1
1	0	0	0	1	0	1	1	1	1	1	0	1	1
1	0	0	0	1	1	1	1	1	0	1	1	1	1
1	0	0	1	0	0	1	1	1	0	1	1	1	1
1	0	0	1	0	1	1	1	0	1	1	1	1	1
1	0	0	1	1	0	1	0	1	1	1	1	1	1
1	0	0	1	1	1	0	1	1	1	1	1	1	1

3.1.4 错误现象及分析

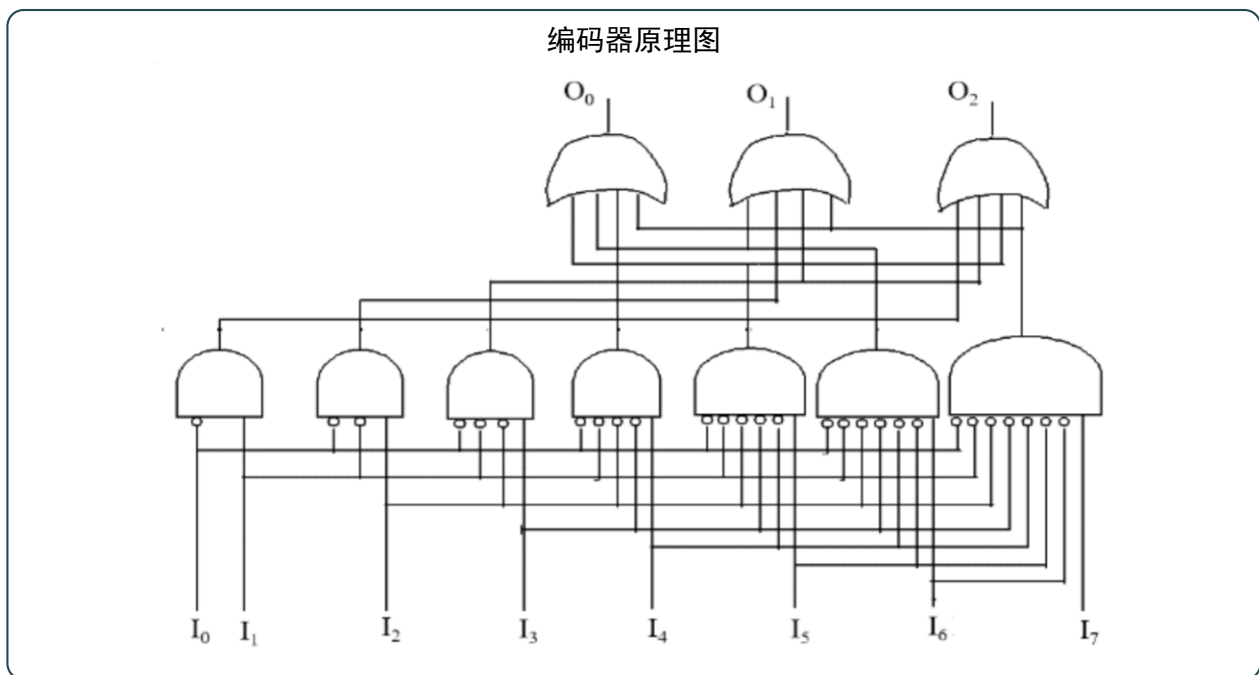
实验中没有发生错误。

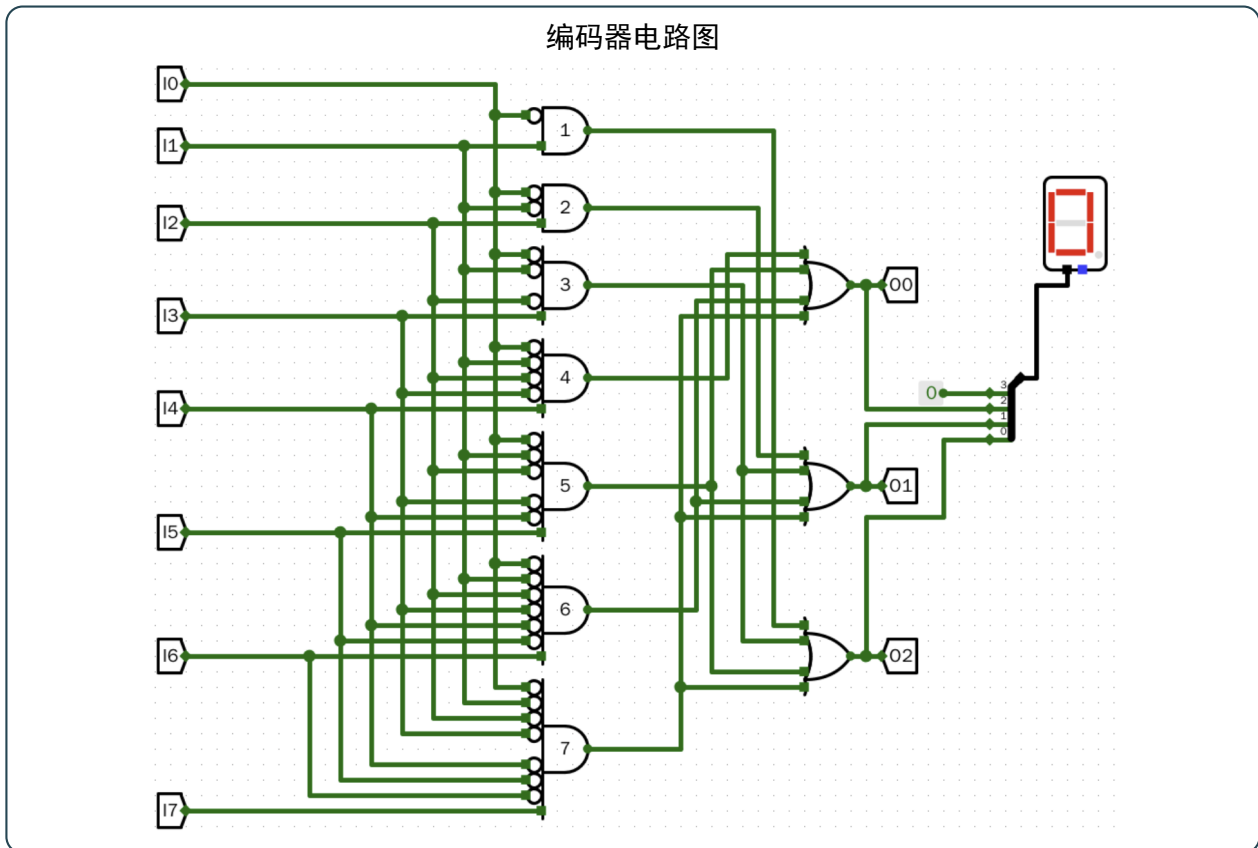
3.2 编码器实验

3.2.1 实验整体方案设计

- 顶层模块设计：实验电路较为简单，不需要顶层模块设计图。
- 输入输出引脚：I₀ -I₇ 为优先编码器输入端，O₀ O₁ O₂ 为输出端 (O₂ 为最低位,O₀ 为最高位)。
- 实现思路：构造优先级判断与编码输出。

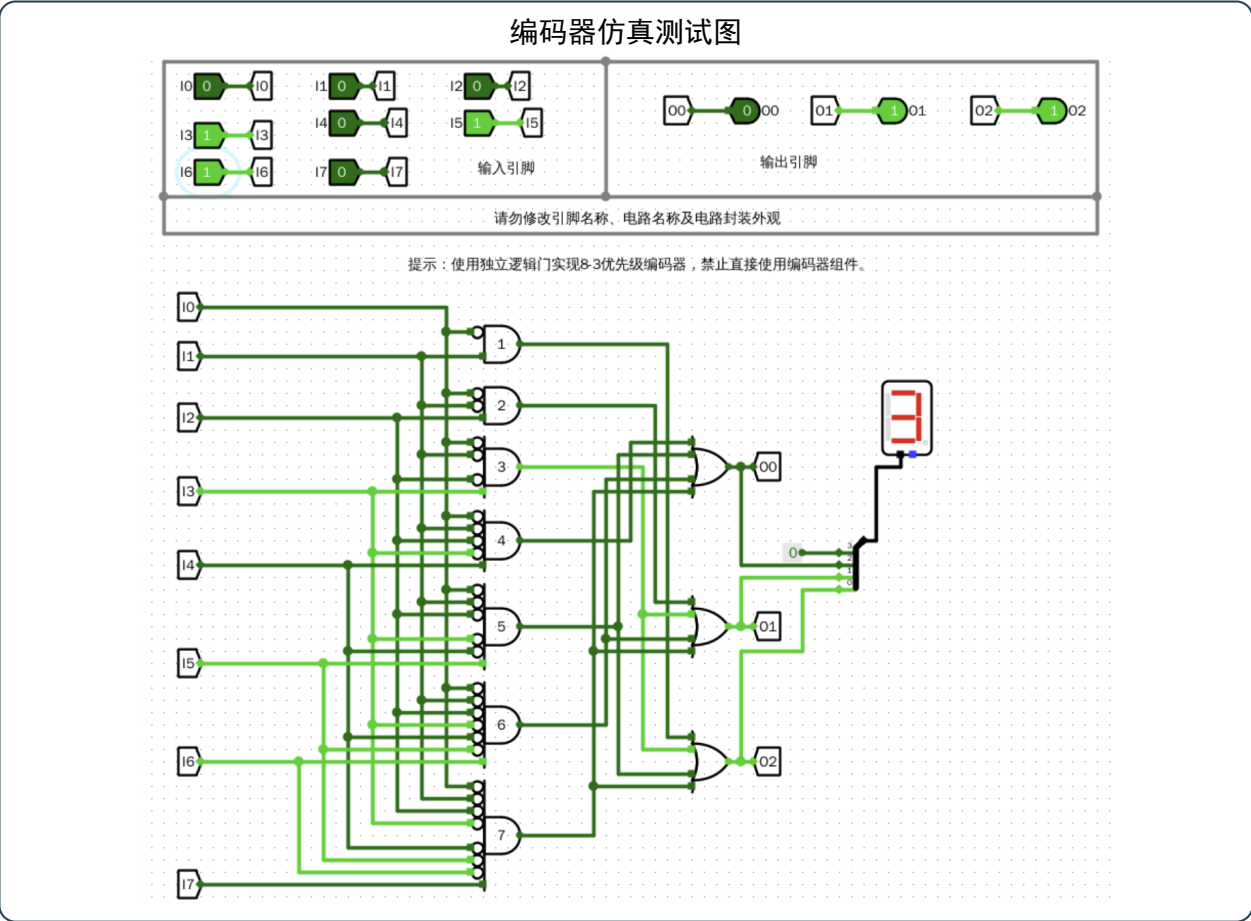
3.2.2 实验原理图和电路图





3.2.3 实验数据仿真测试图与真值表

线上自动测试通过，下面是对某输入的仿真测试截图与真值表。



编码器真值表

输入								输出			Hex 显示
I0	I1	I2	I3	I4	I5	I6	I7	O0	O1	O2	
1	X	X	X	X	X	X	X	0	0	0	0
0	1	X	X	X	X	X	X	0	0	1	1
0	0	1	X	X	X	X	X	0	1	0	2
0	0	0	1	X	X	X	X	0	1	1	3
0	0	0	0	1	X	X	X	1	0	0	4
0	0	0	0	0	1	X	X	1	0	1	5
0	0	0	0	0	0	1	X	1	1	0	6
0	0	0	0	0	0	0	1	1	1	1	7

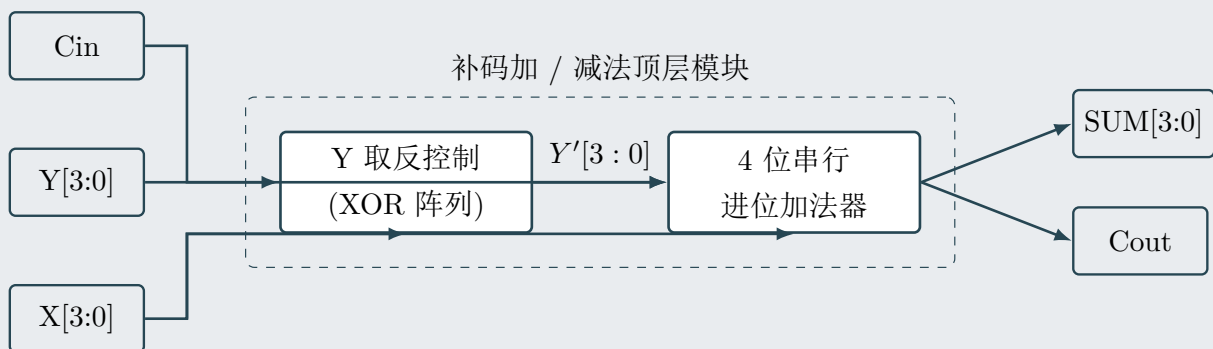
3.2.4 错误现象及分析

实验中没有发生错误。

3.3 加法器与补码减法实验

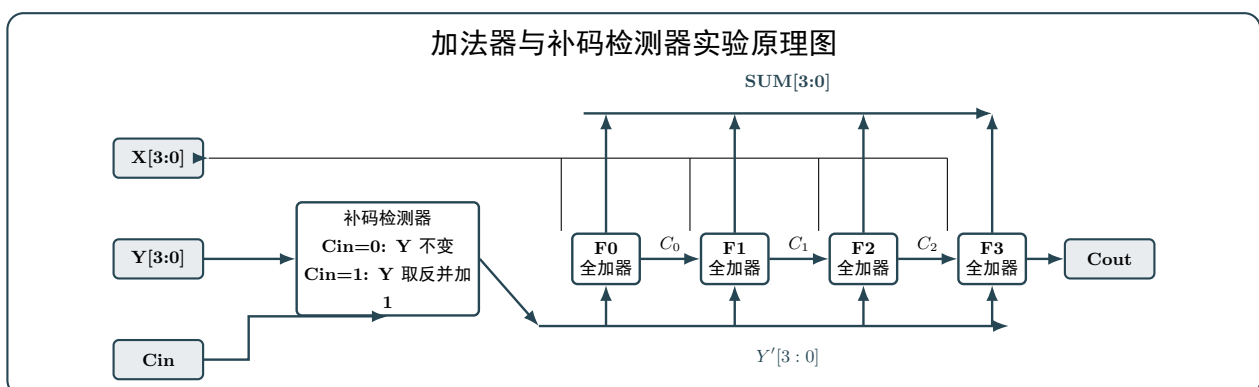
3.3.1 实验整体方案设计

- 顶层模块设计图：见下方图表，先由 Y 与 Cin 结合控制补码，再将 X 与结果送入 4 位串行加法器。
- 输入输出引脚：见下表。
- 实现思路：先实现 1 位全加器，再串联得到 4 位串行进位加法器，并扩展为补码加减法电路。

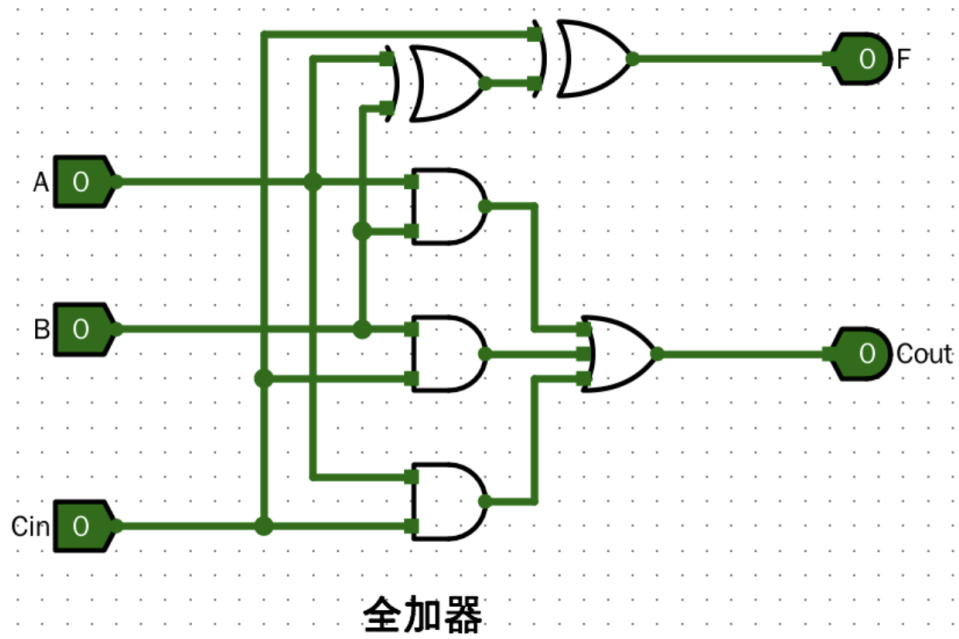


X Y	全加器的两个操作数
Cin	区分加减法运算，Cin=0 执行补码加法，Cin=1 执行补码减法
SUM	全加器的运算结果
Cout	判断 X 与 Y 或者 (-Y) 的补码加法时是否产生溢出进位（若进位则为 1）

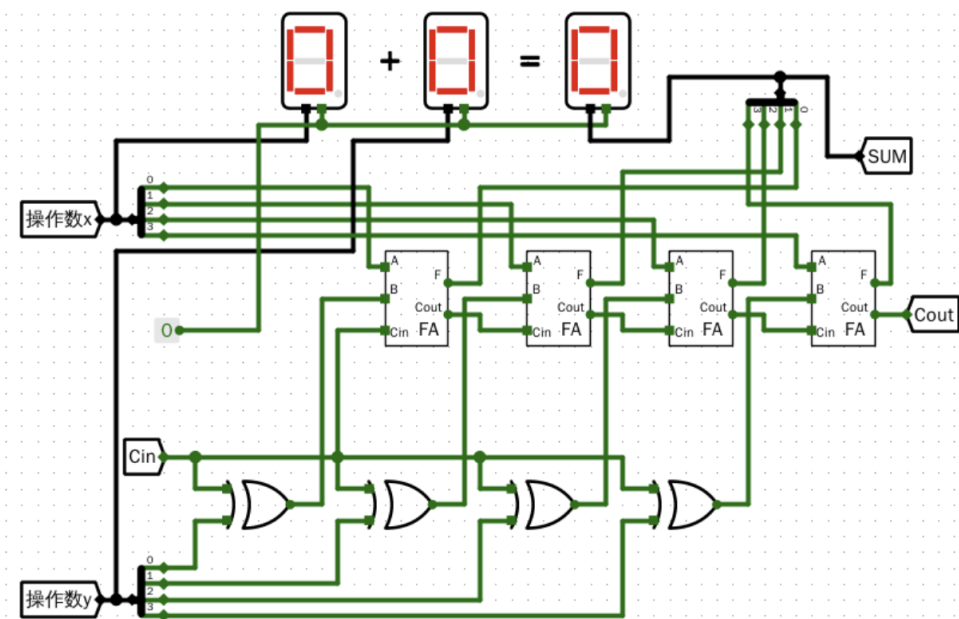
3.3.2 实验原理图和电路图



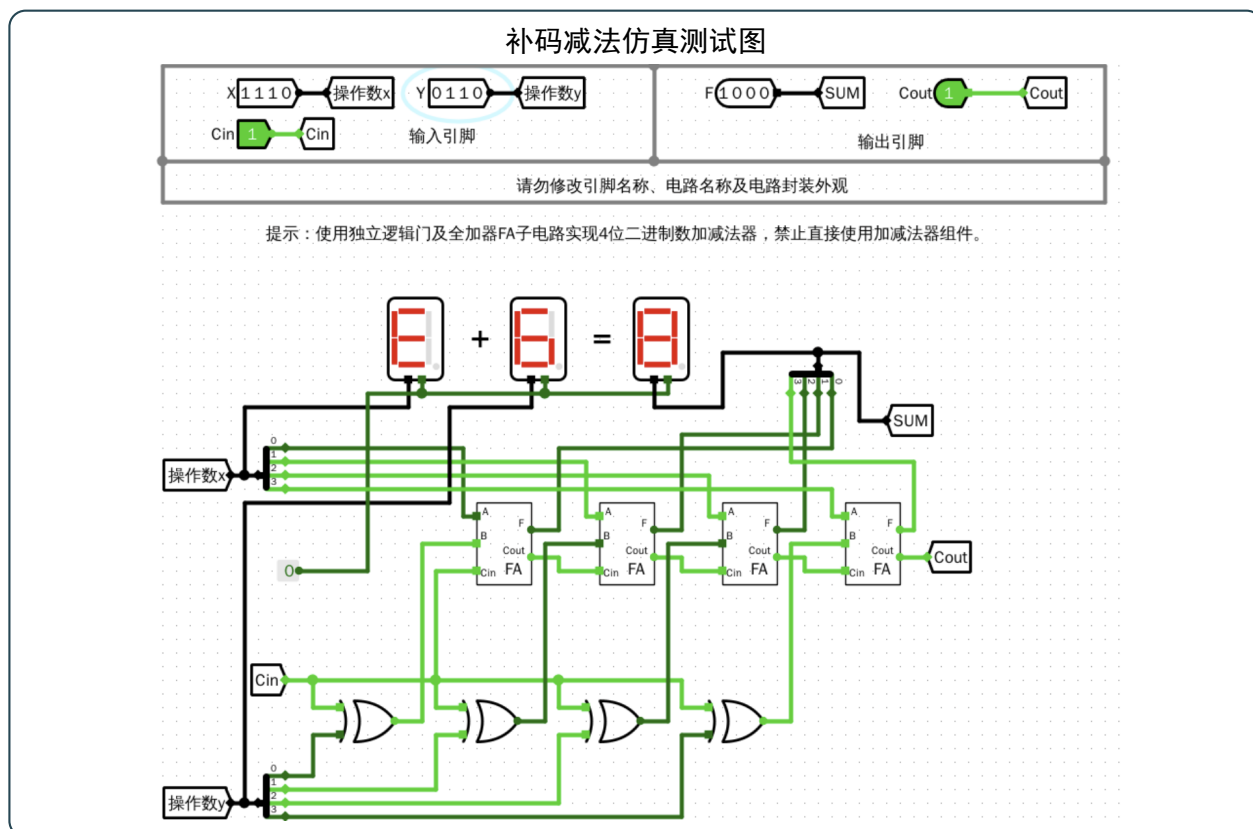
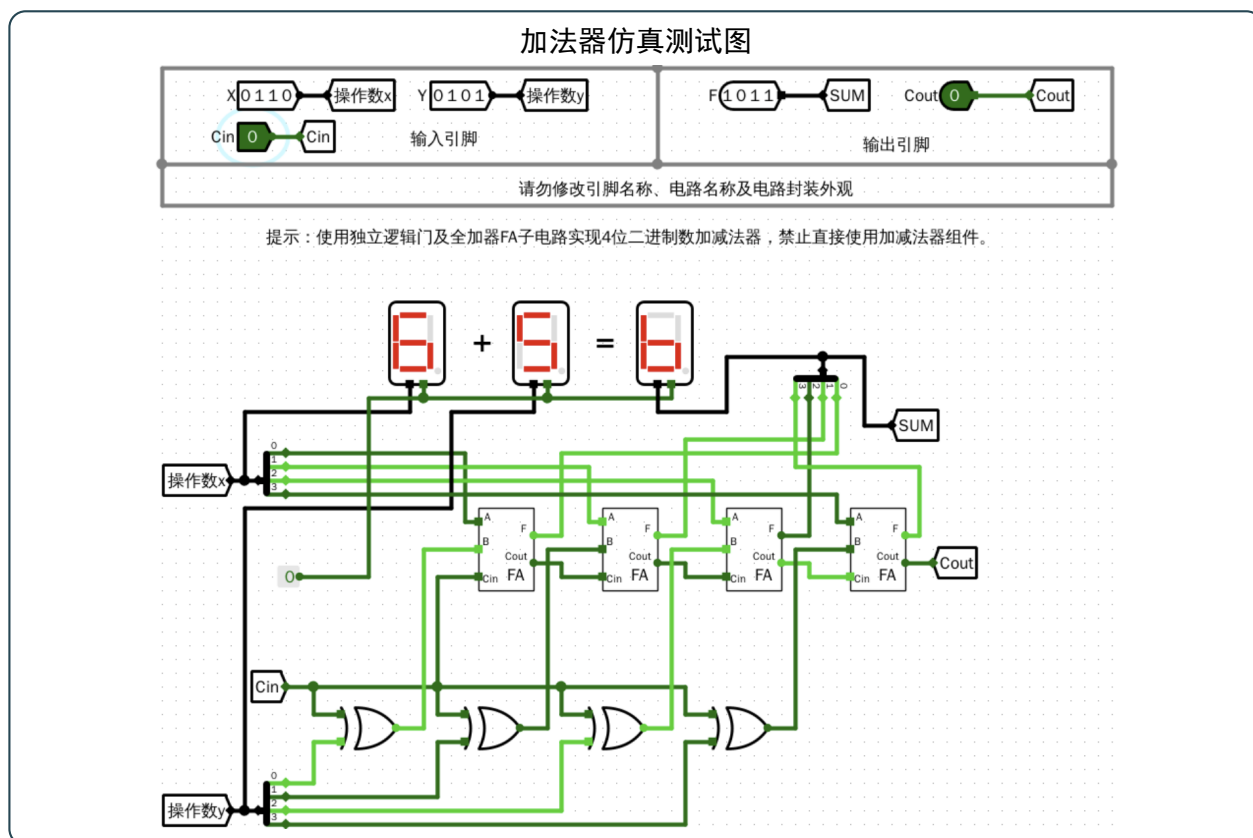
全加器电路图



4 位加法器与补码减法电路图



3.3.3 实验数据仿真测试图



真值表此处过长且不便展示，故省略。

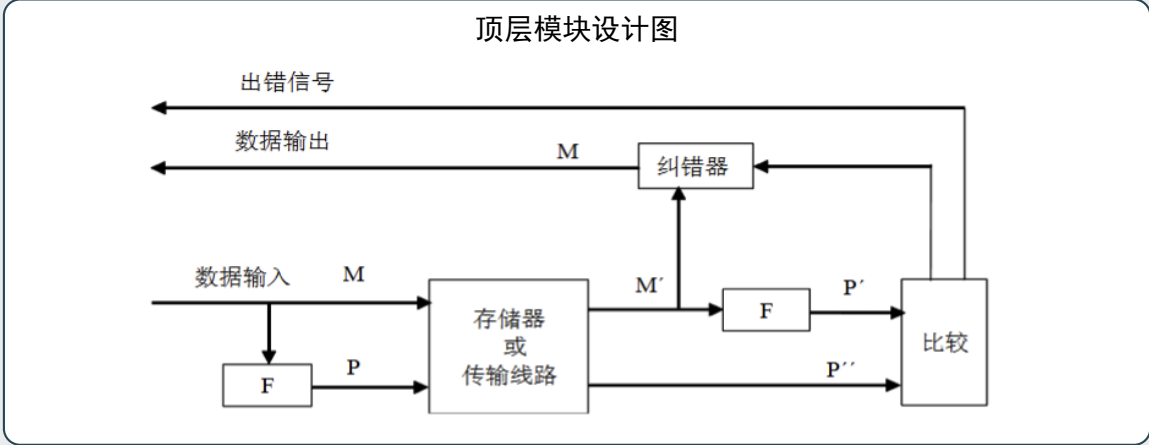
3.3.4 错误现象及分析

实验中没有发生错误。

3.4 汉明码校验电路

3.4.1 实验整体方案设计

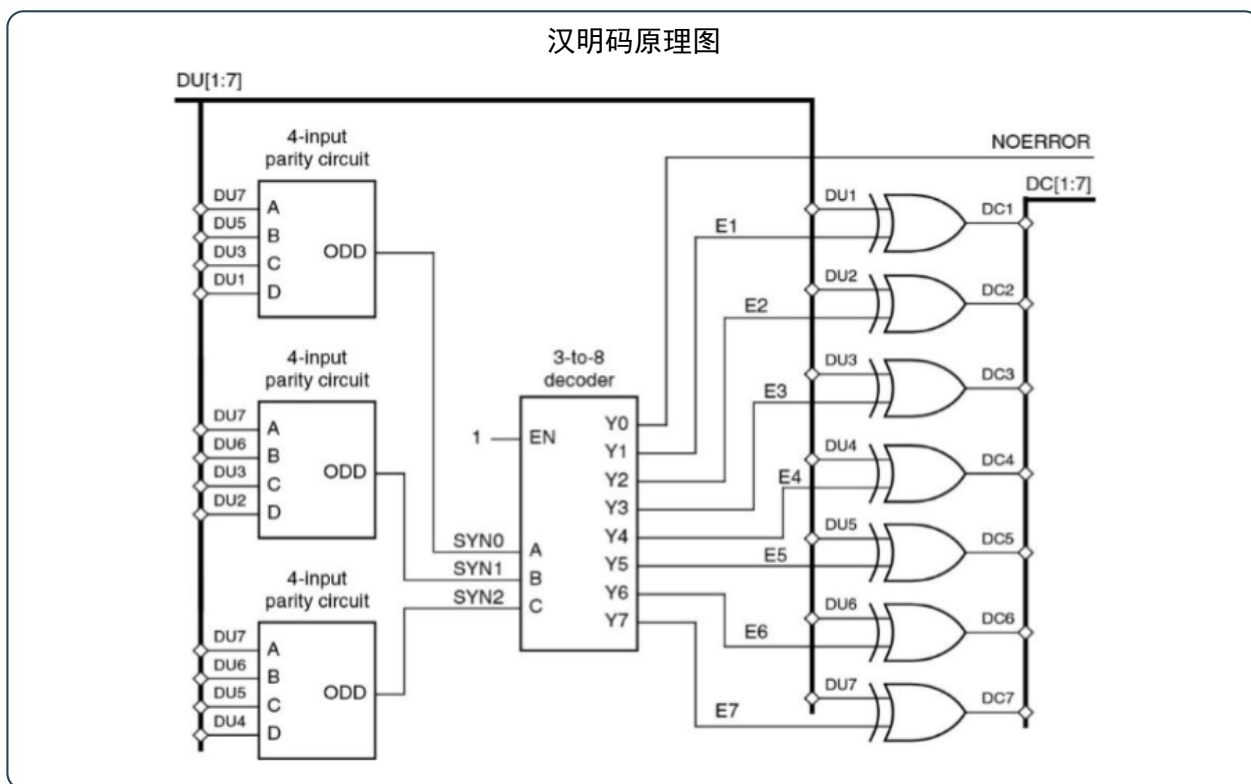
- 顶层模块设计图：



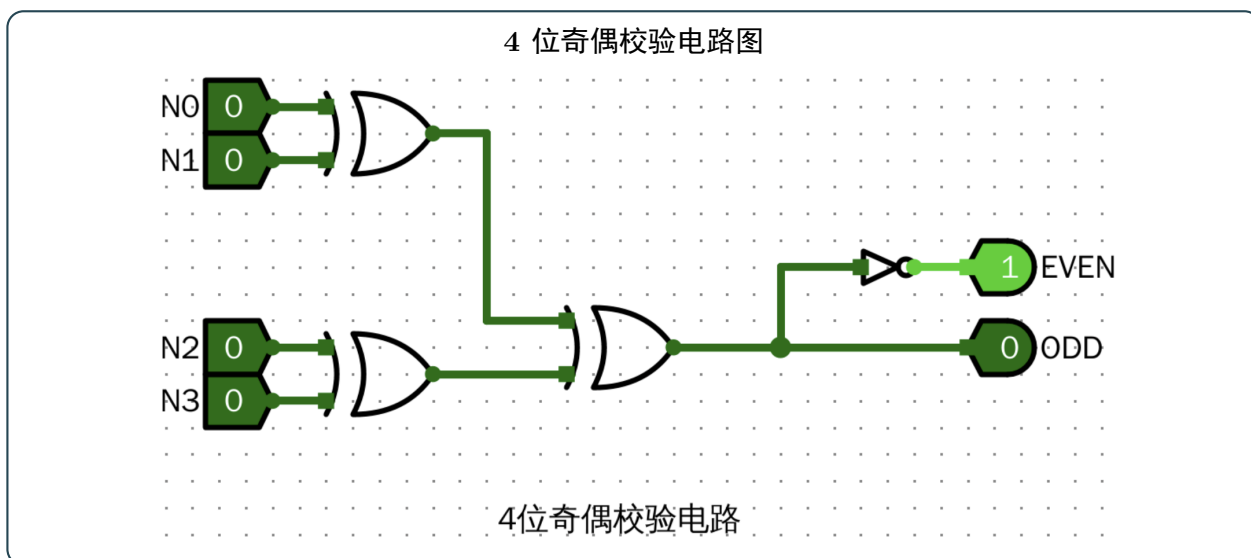
- 输入输出引脚：见下表。
- 实现思路：利用译码器、奇偶校验子电路、隧道和分线器等组件构造检错逻辑。

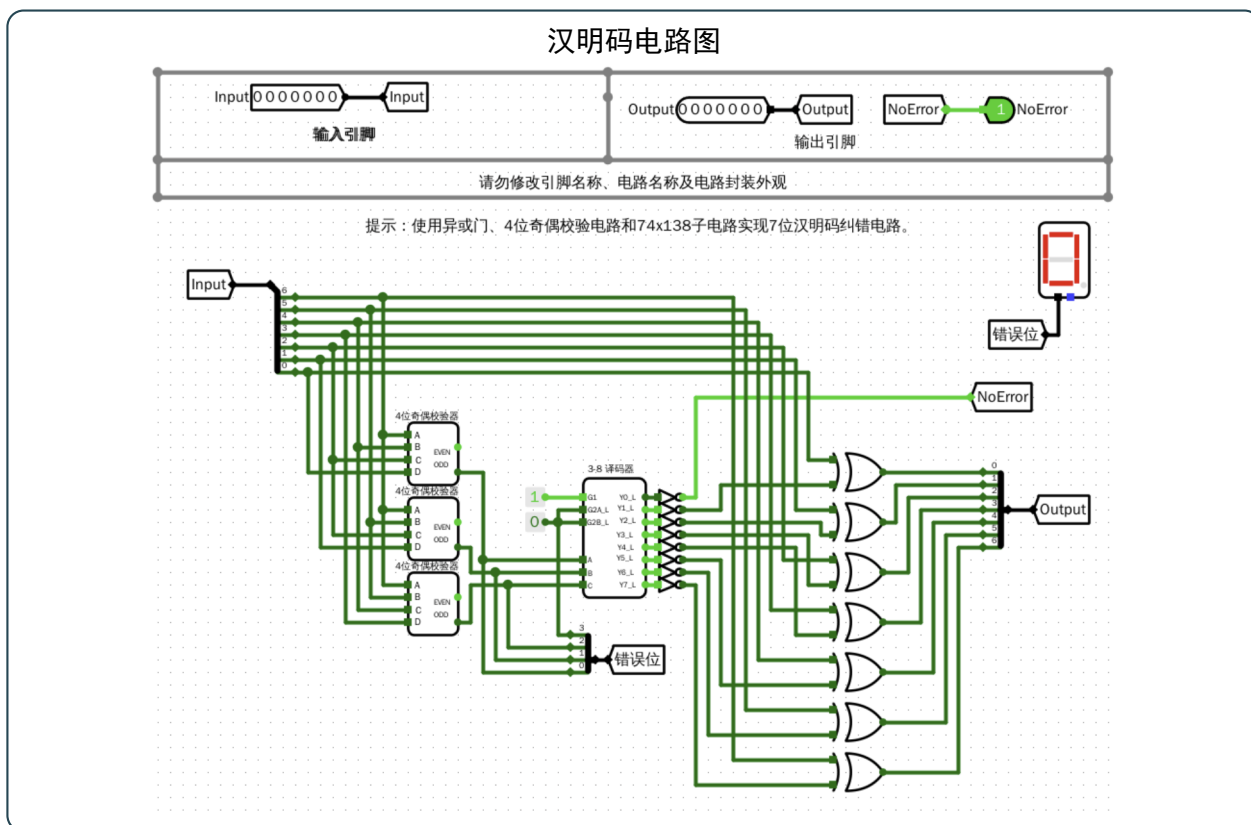
Input	输入需要进行检验/纠错的数据
Output	输出纠错后的数据
NoError	指示数据是否有误（无误为 1）
错误位	指出时数据的哪一位出现错误

3.4.2 实验原理图和电路图

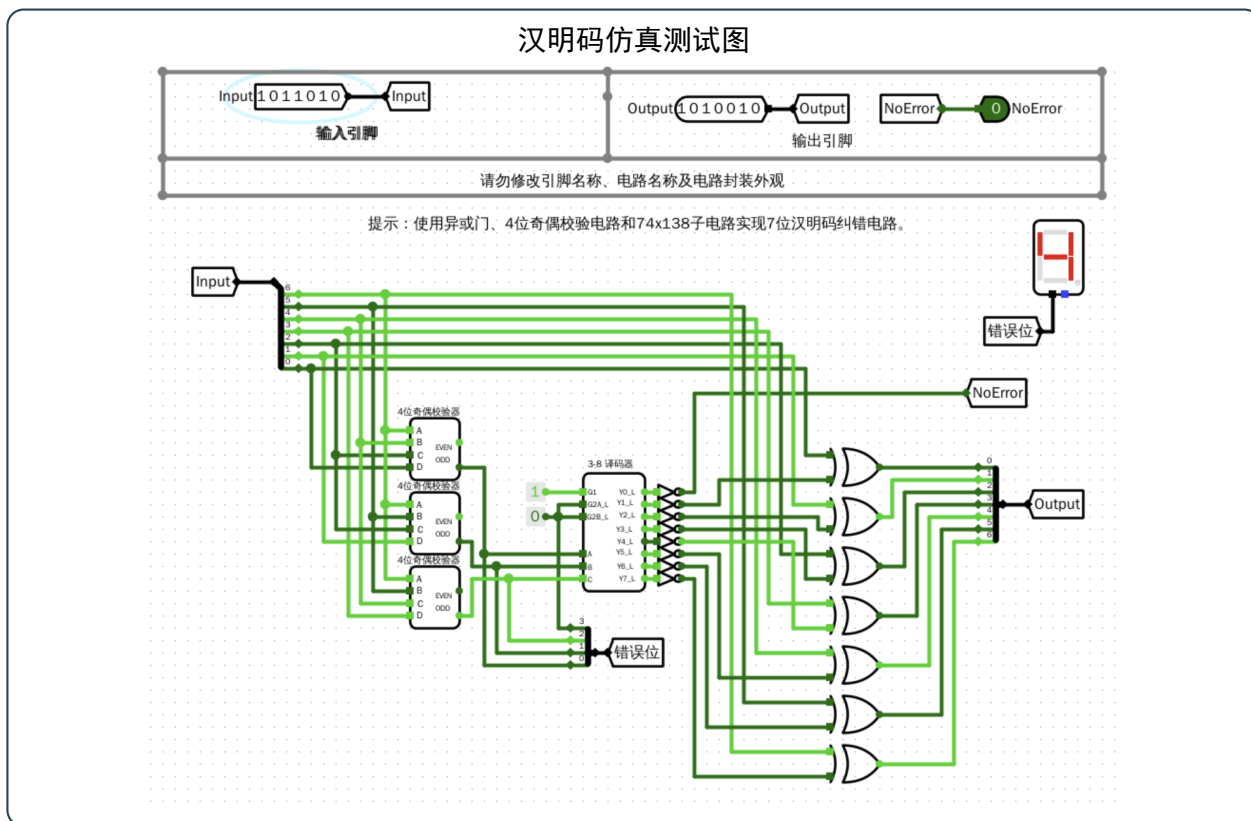


实验中使用的译码器为 2.1 节设计的 3-8 译码器。





3.4.3 实验数据仿真测试图



真值表此处过长且不便展示，故省略。

3.4.4 错误现象及分析

实验中没有发生错误。

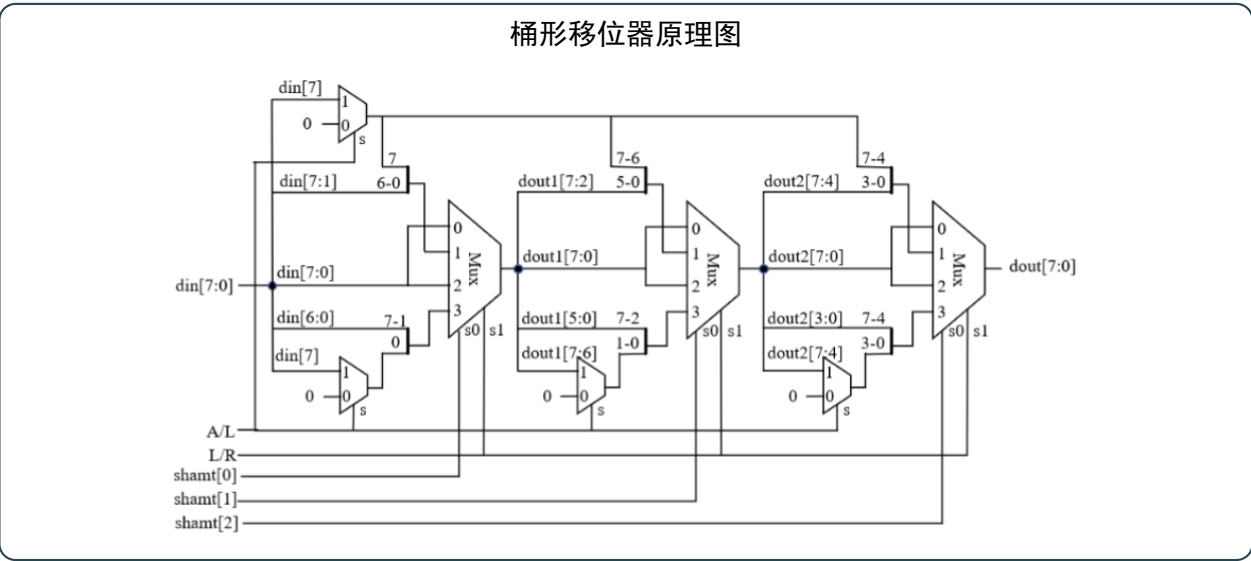
3.5 桶形移位器实验

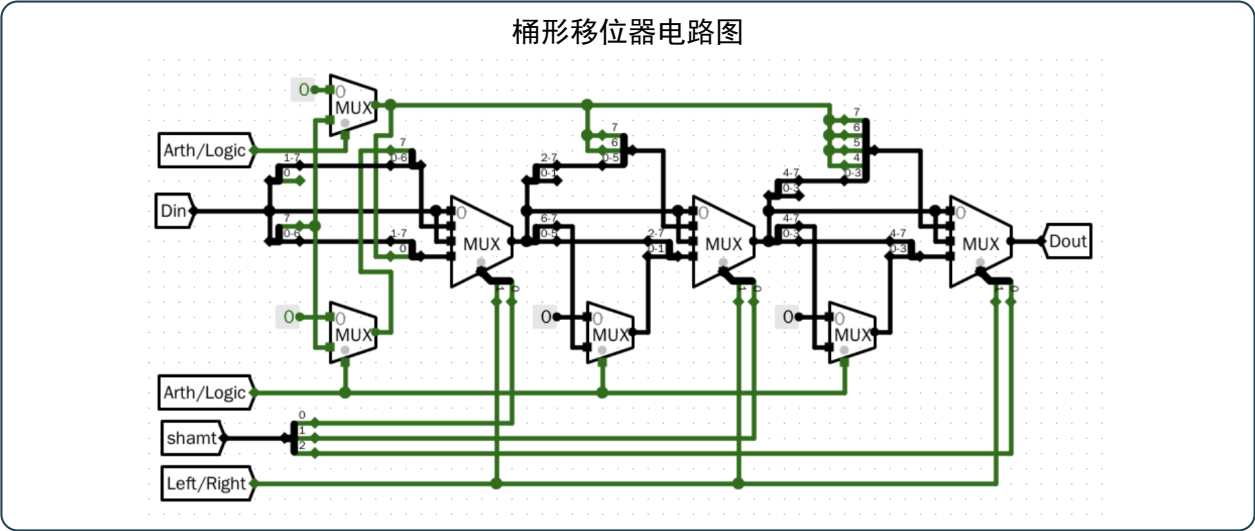
3.5.1 实验整体方案设计

- 顶层模块设计图：本实验电路较为简单，不需要顶层模块设计图。
- 输入输出引脚：见下表。
- 实现思路：使用三级 4 路 8 位多路选择器级联，结合分线器和常量 0 完成移位控制。

Din	输入数据
shamt	控制移位位数，shamt[0]、shamt[1]、shamt[2] 分别控制是否移位 1、2、4 位，均为高电平有效
Left/Right	控制左移/右移，1 为左移，0 为右移
Arth/Logic	控制算数/逻辑移位，置 1 时算数（循环）移位，置 0 时逻辑移位
Dout	输出数据

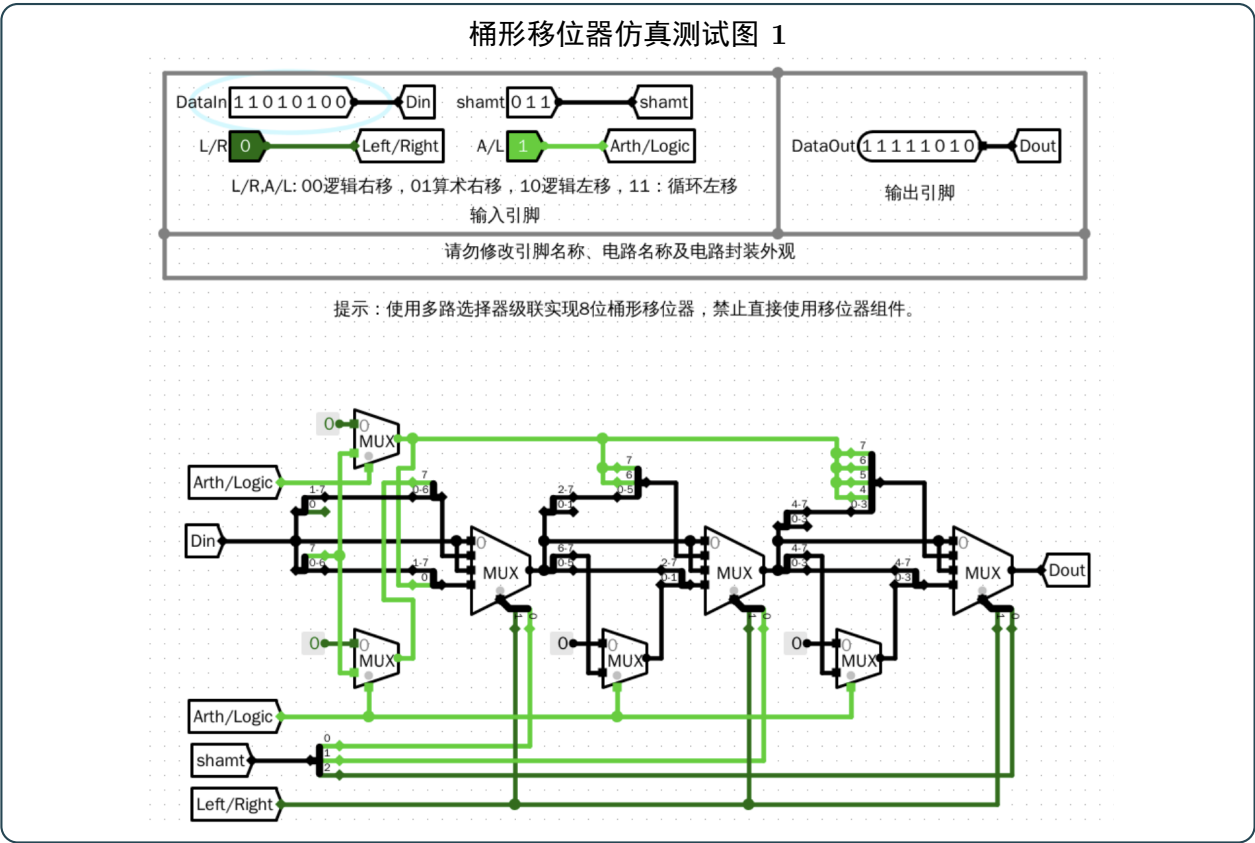
3.5.2 实验原理图和电路图

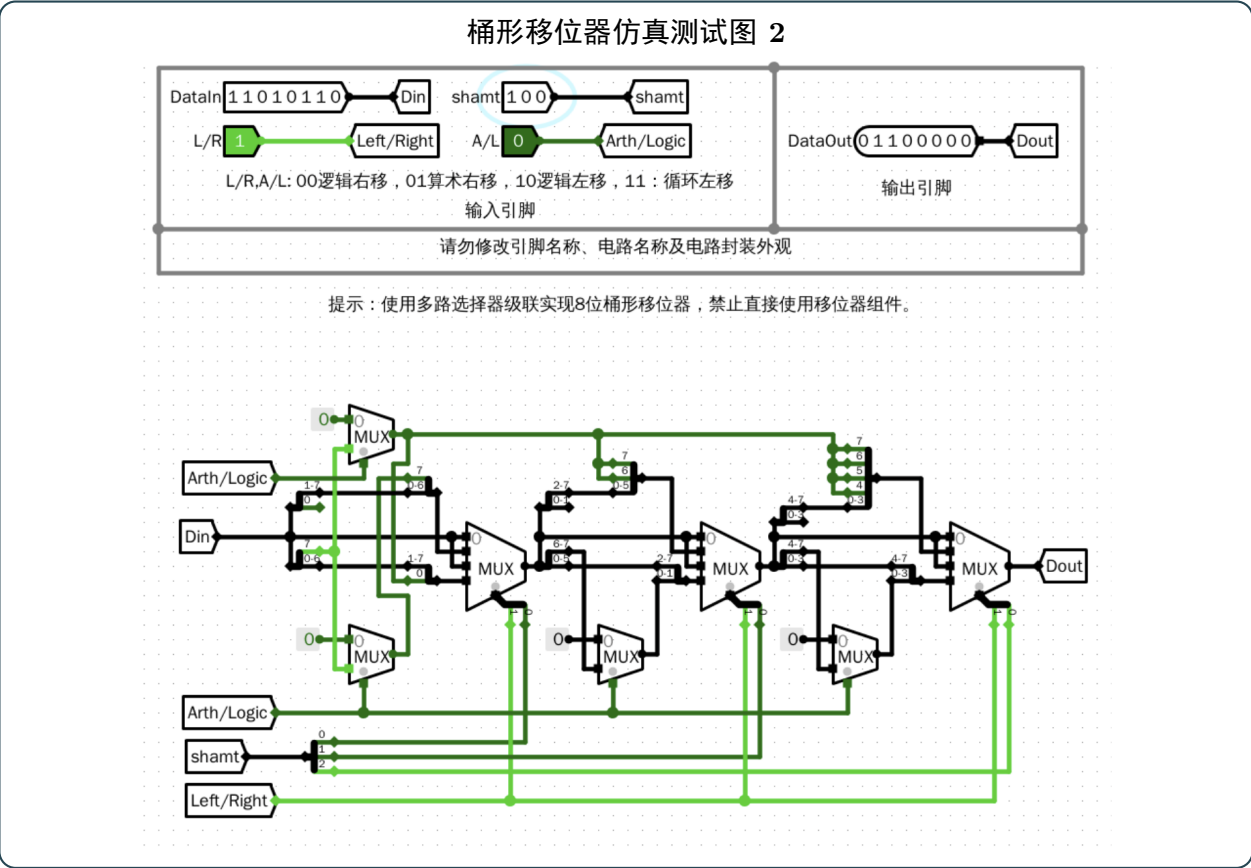




3.5.3 实验数据仿真测试图

可记录不同移位方向、移位位数和移位类型下的输出结果。





Din	shamt	LR	AL	Dout
x	y($1 \leq y \leq 7$, 下同)	0	0	逻辑右移 y 位, 移入 0
x	y	0	1	算数右移 y 位, 移入符号位
x	y	1	0	逻辑左移 y 位, 移入 0
x	y	1	1	循环左移 y 位, 将移出位移入

3.5.4 错误现象及分析

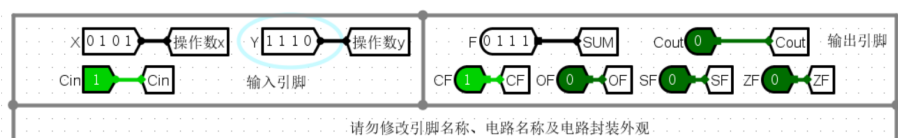
实验中没有发生错误。

4 思考题

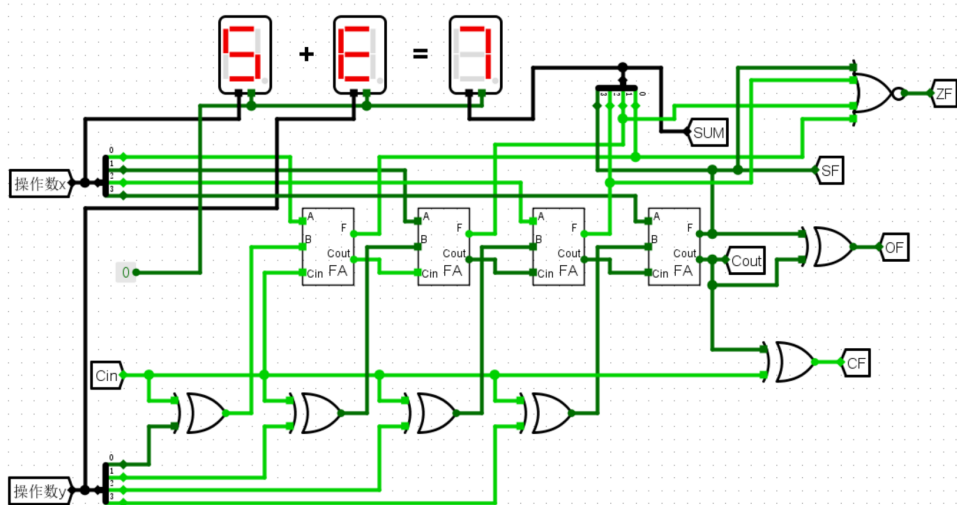
4.1 修改实验中的加法器电路，生成进位标志 CF、溢出标志 OF、符号标志 SF 和结果为 零标志位 ZF。

- **CF (Carry Flag, 进位标志)**: 用于表示无符号运算是否产生进位或借位。对加法运算, 若最高位产生进位则 $CF=1$, 否则 $CF=0$ 。可写为 $CF = C_{out}$ 。
- **OF (Overflow Flag, 溢出标志)**: 用于表示有符号补码运算是否发生溢出。若符号位的进位与最高位的进位不同, 则 $OF=1$; 否则 $OF=0$ 。可写为 $OF = C_n \oplus C_{n-1}$, 其中 C_n 为最高位输出进位, C_{n-1} 为最高位输入进位。
- **SF (Sign Flag, 符号标志)**: 用于表示结果的符号。结果最高位为 1 时表示负数, $SF=1$; 否则 $SF=0$ 。可写为 $SF = S_{n-1}$ 。
- **ZF (Zero Flag, 零标志)**: 用于表示运算结果是否为 0。若所有结果位均为 0, 则 $ZF=1$; 否则 $ZF=0$ 。可写为 $ZF = \overline{S_3} + \overline{S_2} + \overline{S_1} + \overline{S_0}$ 。

带标志位的加法器电路图



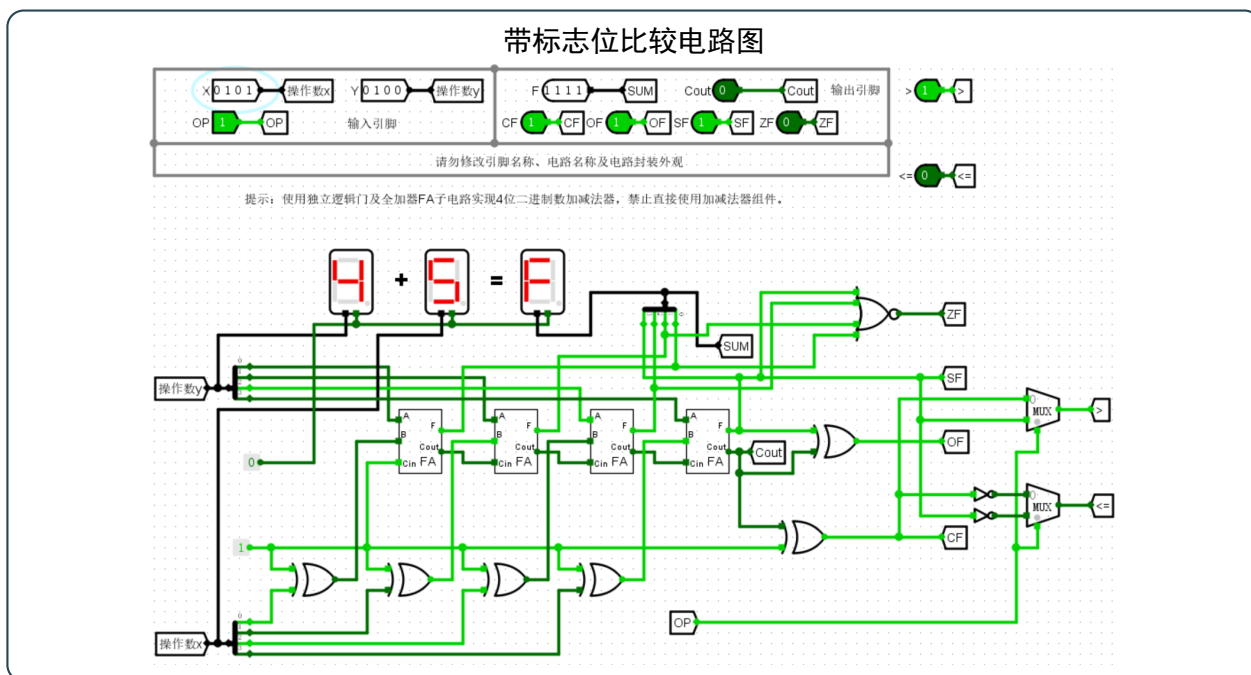
提示: 使用独立逻辑门及全加器FA子电路实现4位二进制数加减法器, 禁止直接使用加减法器组件。



4.2 通过带标志位加法器生成带符号数与不带符号数小于等于比较运算 $X \leq Y$ 的电路。

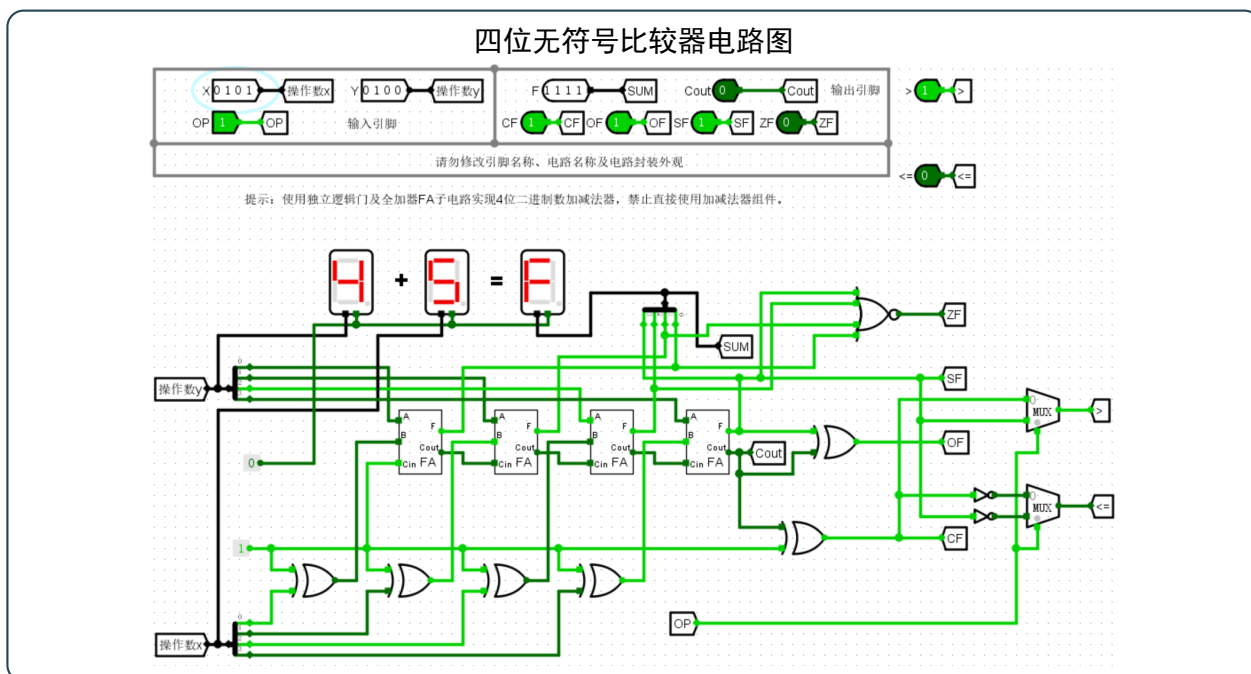
思路: 首先将带标志位加法器的 Cin 值设为常数 1, 进行 $Y - X$ 的运算。然后根据 CF、OF、SF 的值判断 $X \leq Y$ 的结果。

- 若为有符号数, 则 $SF = 1$ 时被减数小于减数, $SF = 0$ 时被减数大于减数;
- 若为无符号数, 则 $CF = 1$ 时被减数小于减数, $CF = 0$ 时被减数大于减数。



4.3 使用逻辑门电路实现两个四位无符号二进制数比较器，输出大于、等于两个运算结果。

逐位比较即可。



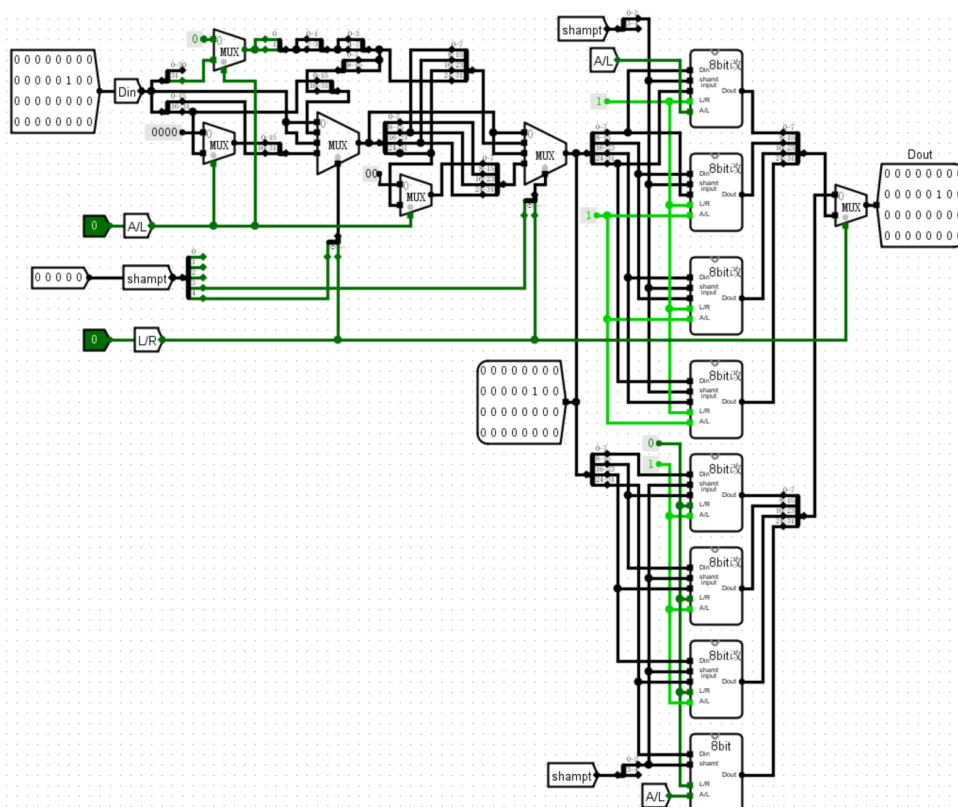
4.4 如何使用 8 位桶形移位器扩展到 32 位桶形移位器。

在原有 8 位桶形移位器的基础上，可以先将其扩展为一个支持级联的 8 位模块，再通过多个模块并联组合实现 32 位桶形移位器。具体做法是为每个 8 位移位单元增加一个额外输入端 input，用来接收相邻字节的补充数据，从而避免移位后高位或低位直接补零。

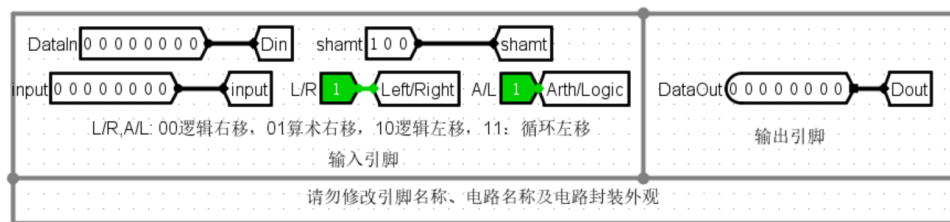
当 $AL=0$ 时，移位器仍按照普通逻辑移位方式工作：左移时低位补 0，右移时位补 0；当 $AL=1$ 时，移位器进入扩展模式：左移时由 input 的高位补入 datain 的低位，右移时由 input 的低位补入 datain 的高位。这样，每个 8 位模块都可以从相邻模块获得所需的数据位，最终实现 32 位范围内的整体移位。

在 32 位扩展中，可将 4 个 8 位移位模块按字节级联，并结合 5 位移位控制信号完成 1、2、4、8、16 位的分级控制。低位模块负责输出结果的最低字节，高位模块负责接收来自更高或更低字节的补充位。通过这种“分级移位 + 字节串联”的方式，既能保持原有 8 位桶形移位器的设计结构，又能在较少修改的前提下扩展到 32 位数据宽度。

32 位桶形移位器扩展示意图



修改后的 8 位桶形移位器扩展示意图



提示：使用多路选择器级联实现8位桶形移位器，禁止直接使用移位器组件。

